

astro-one V0.2.0

航天 AI 智能体框架使用说明书

空间目标感知国家重点实验室

<https://github.com/astro-one/astro-one>

2026 年 5 月

版本 1.0

开发者：空间目标感知国家重点实验室（State Key Laboratory of Space
Target Perception）

许可证：MIT License

Python 版本：≥ 3.11

发布日期：2026 年 3 月

目录

1	什么是 astro-one	4
1.1	核心定位	4
1.2	技术栈	4
2	项目架构	5
2.1	整体数据流	5
2.2	模块结构	5
3	版本特性（V0.2.0）	6
4	环境要求	6
5	安装方式	6
5.1	PyPI 安装（推荐）	6
5.2	源码安装	7
5.3	Docker 安装	7
6	初始化配置	7
6.1	配置文件位置	8
7	5 分钟上手	8
7.1	步骤 1：初始化	8
7.2	步骤 2：配置 LLM Provider	8
7.3	步骤 3：运行 Agent	9
7.4	步骤 4：启动 Gateway	9
8	本地模型设置（Ollama）	9
9	Python API 使用	10
10	全局选项	11
11	onboard —初始化向导	11
12	agent —Agent 对话	11
12.1	交互模式快捷键	12
13	gateway —启动网关	12
14	status —状态查看	13

15 channels login	—渠道登录	13
16 provider login	—Provider 登录	13
17	配置文件结构	14
18 agents.defaults	—Agent 默认配置	14
19 providers	—LLM 提供商配置	15
20 modelPresets	—模型预设	16
21 channels	—渠道配置	16
22 tools	—工具配置	17
23	环境变量	18
24 MLF	—轨道机动检测	19
24.1	功能概述	19
24.2	Agent 工具名	20
24.3	输入数据格式	20
24.4	输出格式	20
24.5	使用示例	20
24.6	军事分析输出	20
25 IOD	—轨道初定	21
25.1	功能概述	21
25.2	Agent 工具名	21
25.3	输入数据格式	21
25.4	输出格式	21
25.5	军事分析输出	21
26 Orbin	—轨道根数预测	21
26.1	功能概述	21
26.2	Agent 工具名	22
26.3	输入数据格式	22
26.4	输出格式	22
26.5	军事分析输出	22
27	航天工具联合使用	22
28	渠道概述	23

29 飞书渠道配置详解	23
29.1 飞书特有功能	24
30 渠道开发	24
31 提供商匹配机制	25
32 常用提供商配置示例	25
32.1 OpenAI	25
32.2 Anthropic Claude	25
32.3 本地 Ollama	26
32.4 AWS Bedrock	26
32.5 DeepSeek	27
33 备选模型 (Fallback)	27
34 状态机处理流程	28
35 上下文构建	28
36 记忆系统	29
36.1 Dream 记忆处理器	29
37 子 Agent (SubAgent)	29
38 会话管理	30
39 内置工具列表	30
40 Shell 工具详解	30
41 MCP 协议支持	31
42 工具扩展	31
43 SSRF 防护	32
44 Workspace 隔离	32
45 命令注入防护	32
46 访问控制	33
47 快速部署	33

48 docker-compose.yml 结构	33
49 多实例部署	34
50 技能概述	34
51 技能目录结构	34
52 SKILL.md 格式	35
53 内置技能	35
54 OpenAI 兼容 API	35
54.1 配置	36
54.2 端点	36
54.3 调用示例	36
55 AeroBench — 航天垂直领域基准测评	36
55.1 运行方式	36
55.2 测评任务	37
56 安装问题	37
57 配置问题	38
58 运行时问题	38
59 渠道问题	39
60 航天工具问题	39
A 生产环境配置参考	39

项目概述

1 什么是 astro-one

astro-one 是一个超轻量级的航天 AI 智能体（Agent）框架，基于 HKUDS 团队的 nanobot 构建。它提供核心 Agent 功能，支持多种大语言模型（LLM）提供商和即时通讯渠道，特别针对**航天垂直领域**（空间态势感知、轨道分析）做了深度定制。

1.1 核心定位

- **航天 AI 助手**：内置轨道机动检测（MLF）、初始轨道确定（IOD）、轨道根数预测（Orbin）三大航天推理工具。
- **多渠道接入**：支持飞书、Telegram、Discord、QQ、微信、Slack、WhatsApp 等 15+ 聊天平台。
- **多模型兼容**：支持 OpenAI、Anthropic、Google Gemini、DeepSeek、Ollama 等 30+ LLM 提供商。
- **轻量可扩展**：基于插件架构（Channel / Tool / Skill 三层扩展），核心代码精炼。

1.2 技术栈

- **语言**：Python 3.11+
- **异步框架**：asyncio
- **LLM 接入**：LiteLLM + 原生 SDK（OpenAI、Anthropic、boto3 等）
- **配置管理**：Pydantic v2 + Pydantic-Settings
- **CLI**：Typer + Rich + prompt_toolkit
- **消息队列**：asyncio.Queue（内置轻量总线）
- **会话存储**：JSONL 格式本地持久化

2 项目架构

2.1 整体数据流



2.2 模块结构

模块	路径	功能
Agent 核心	astro_one/agent/	主循环、上下文构建、记忆系统、子 Agent
消息总线	astro_one/bus/	发布/订阅模式的异步消息路由
LLM 提供商	astro_one/providers/	30+ LLM 后端统一接口
聊天渠道	astro_one/channels/	15+ 聊天平台接入
工具系统	astro_one/agent/tools/	Shell、文件、Web、MCP、航天工具
会话管理	astro_one/session/	JSONL 格式会话持久化
配置系统	astro_one/config/	Pydantic v2 配置模型
CLI 界面	astro_one/cli/	Typer 命令行界面
技能系统	astro_one/skills/	可扩展的 Agent 技能
安全模块	astro_one/security/	SSRF 防护、网络隔离
API 服务	astro_one/api/	OpenAI 兼容 REST API

3 版本特性（V0.2.0）

- **Agent 核心:** 完整的状态机驱动处理流程（RESTORE → COMPACT → COMMAND → BUILD → RUN → SAVE → RESPOND → DONE）
- **流式输出:** 支持 SSE 和即时通讯渠道的增量输出
- **记忆系统:** 三级记忆架构（MemoryStore / Consolidator / Dream）
- **会话管理:** JSONL 持久化，自动压缩与恢复
- **航天工具:** MLF 轨道机动检测、IOD 轨道初定、Orbin 轨道预测
- **安全防护:** SSRF 防护、workspace 隔离、命令注入检测
- **模型预设:** 支持多套模型配置的快速切换
- **Web UI:** WebSocket 驱动的实时 Web 界面
- **子 Agent:** 支持后台异步子任务执行
- **Docker 支持:** 开箱即用的容器化部署
安装与环境配置

4 环境要求

组件	最低版本	推荐配置
Python	3.11	3.12+
操作系统	Linux / macOS / Windows	Ubuntu 22.04+
内存	2 GB	8 GB+（运行本地模型时）
磁盘	500 MB	2 GB+（含模型文件）
网络	出站 HTTPS	稳定互联网连接

表 2: 环境要求

5 安装方式

5.1 PyPI 安装（推荐）

```
# 基础安装
pip install astro-one-ai

# 带开发依赖
pip install "astro-one-ai[dev]"

# 带特定渠道依赖
```

```
pip install "astro-one-ai[wecom,matrix]"
```

5.2 源码安装

```
git clone https://github.com/astro-one/astro-one.git
cd astro-one

# 开发模式安装
pip install -e .

# 全量安装
pip install -e ".[dev,wecom,matrix]"
```

5.3 Docker 安装

```
# 使用 docker-compose (推荐)
docker compose up -d astro-one-gateway

# 手动构建
docker build -t astro-one .
docker run -d --name astro-one -p 18790:18790 astro-one
```

6 初始化配置

安装完成后，运行初始化向导：

```
astroone onboard
```

此命令将：

1. 创建配置目录 `~/.astro_one/`
2. 生成默认配置文件 `config.json`
3. 创建工作空间目录 `~/.astro_one/workspace/`
4. 生成模板文件（`SOUL.md`、`AGENTS.md`、`USER.md`）
5. 初始化 Git 仓库（用于记忆版本管理）

6.1 配置文件位置

文件/目录	默认路径
配置文件	~/.astro_one/config.json
工作空间	~/.astro_one/workspace/
会话文件	~/.astro_one/workspace/sessions/
记忆文件	~/.astro_one/workspace/memory/
CLI 历史	~/.astro_one/cli_history

表 3: 默认文件路径

提示

可以使用环境变量 NANOBOT__<section>__<key> 覆盖配置。例如，设置默认模型：

```
export NANOBOT__AGENTS__DEFAULTS__MODEL="openai/gpt-4o"
```

快速入门

7 5 分钟上手

7.1 步骤 1：初始化

```
astroone onboard
```

按提示完成初始化，选择默认模型和提供商。

7.2 步骤 2：配置 LLM Provider

编辑 ~/.astro_one/config.json，添加 API 密钥：

```
{
  "providers": {
    "openai": {
      "apiKey": "sk-xxxxxx",
      "apiBase": "https://api.openai.com/v1"
    },

```

```
"anthropic": {
  "apiKey": "sk-ant-xxxxxx"
},
"agents": {
  "defaults": {
    "model": "anthropic/claude-opus-4-5"
  }
}
```

7.3 步骤 3: 运行 Agent

```
# 单次对话
astroone agent -m " 介绍一下低地球轨道的特点"

# 交互式对话
astroone agent

# 流式输出
astroone agent --stream
```

7.4 步骤 4: 启动 Gateway

```
astroone gateway
```

Gateway 将加载所有已启用的聊天渠道，等待消息接入。

8 本地模型设置（Ollama）

```
# 1. 安装 Ollama
curl -fsSL https://ollama.com/install.sh | sh

# 2. 拉取模型
ollama pull qwen3.5:27b

# 3. 配置 astro-one
```

```
# 编辑 config.json:
```

```
{
  "agents": {
    "defaults": {
      "model": "qwen3.5:27b",
      "provider": "ollama"
    }
  },
  "providers": {
    "ollama": {
      "apiBase": "http://localhost:11434"
    }
  }
}
```

```
# 4. 启动
```

```
astroone agent -m " 你好，请做自我介绍"
```

9 Python API 使用

```
# sdk_example.py
from astro_one.astro_one import AstroOne

# 从配置文件创建实例
bot = AstroOne.from_config()

# 同步运行（在 asyncio 事件循环中）
import asyncio

async def main():
    result = await bot.run(
        " 分析 tools/iod/data/demo_input.csv 中的观测数据",
        session_key="my_session"
    )
    print(f" 回复: {result.content}")
    print(f" 使用工具: {result.tools_used}")
```

```
asyncio.run(main())
```

CLI 命令参考

astro-one 提供两个等价的命令行入口：astro-one 和 astroone。

10 全局选项

选项	说明	默认值
--config	指定配置文件路径	~/.astro_one/config.json
--workspace	覆盖工作空间路径	配置文件中的值
--help / -h	显示帮助信息	—

表 4: 全局选项

11 onboard —初始化向导

```
astroone onboard [OPTIONS]
```

选项	说明	默认值
--config PATH	配置文件写入路径	默认路径
--workspace PATH	工作空间目录	默认路径

表 5: onboard 命令选项

12 agent —Agent 对话

```
# 单次对话
astroone agent -m " 消息内容"

# 交互式对话
astroone agent

# 流式交互
```

```
astroone agent --stream

# 使用特定模型和会话
astroone agent -m " 消息" --session my-session --model
↪ "openai/gpt-4o"
```

选项	说明	默认值
-m / --message	单次消息内容	—（交互模式）
--stream	启用流式输出	false
--model	覆盖默认模型	配置文件中的值
--session	会话标识符	cli:direct
--no-markdown	禁用 Markdown 渲染	false
--thinking	显示模型推理过程	false

表 6: agent 命令选项

12.1 交互模式快捷键

输入	操作
exit / quit / :q	退出交互模式
/new	开始新会话
/model	列出可用模型
/model <name>	切换到指定模型
/models	列出所有模型预设
Ctrl+C	中断当前生成
Enter	发送消息

表 7: 交互模式命令

13 gateway —启动网关

```
astroone gateway [OPTIONS]
```

选项	说明	默认值
--host	绑定地址	0.0.0.0
--port	绑定端口	18790

表 8: gateway 命令选项

Gateway 启动后将：

1. 初始化所有已启用的聊天渠道
2. 启动消息总线的出站分发器
3. 启动 Agent 处理循环
4. 启动 Web UI（如已配置）
5. 启动定时任务调度器（如已配置）

14 status —状态查看

```
astroone status
```

显示：

- 当前配置概览
- 启用的渠道列表及运行状态
- Agent 工作空间路径
- 活跃会话数

15 channels login —渠道登录

```
# 登录 WhatsApp
astroone channels login --provider whatsapp

# 登录 Telegram
astroone channels login --provider telegram
```

16 provider login —Provider 登录

```
# 登录 OpenAI Codex
astroone provider login --provider openai_codex
```


配置参考

17 配置文件结构

配置文件为 JSON 格式，支持 camelCase 键名。完整结构如下：

```
{
  "agents": { ... },           // Agent 配置
  "channels": { ... },         // 聊天渠道配置
  "providers": { ... },        // LLM 提供商配置
  "tools": { ... },            // 工具配置
  "gateway": { ... },          // 网关配置
  "api": { ... },              // API 服务配置
  "modelPresets": { ... }      // 模型预设
}
```

18 agents.defaults — Agent 默认配置

字段 (camelCase)	默认值	说明
workspace	~/.astro_one/workspace	工作空间路径
model	anthropic/claude-opus-1.4	默认模型标识符
provider	"auto"	模型提供商（auto 表示自动匹配）
modelPreset	null	默认模型预设名称
maxTokens	8192	最大输出 token 数
contextWindowTokens	65536	上下文窗口大小
temperature	0.1	生成温度
maxToolIterations	200	单轮最大工具调用次数
maxConcurrentSubagents	4	最大并发子 Agent 数
maxToolResultChars	6000	工具结果最大字符数
providerRetryMode	"standard"	重试模式（standard/persistent）
toolHintMaxLength	40	工具提示最大长度
reasoningEffort	null	推理深度（null/low/medium/high）
timezone	"UTC"	Agent 所在时区
botName	"astro-one"	机器人名称
botIcon	"[rocket]"	机器人图标
unifiedSession	false	是否跨渠道共享会话
disabledSkills	[]	禁用的技能列表

sessionTtlMinutes	0	空闲会话自动压缩时间（0= 禁用）
maxMessages	120	会话最大保留消息数
consolidationRatio	0.5	记忆巩固目标比例
fallbackModels	[]	备选模型列表

19 providers —LLM 提供商配置

支持的提供商列表（每个均为 {"apiKey": "...", "apiBase": "..."} 格式）:

配置键	后端类型	说明
openai	OpenAI 原生	GPT-4o, GPT-4.1, o3 等
anthropic	Anthropic 原生	Claude Opus 4.7, Sonnet 4.6, Haiku 4.5
bedrock	AWS Bedrock	通过 AWS SDK 访问
azure_openai	Azure OpenAI	Azure 托管的 OpenAI 服务
gemini	Google Gemini	Gemini 2.5 Pro/Flash 等
deepseek	DeepSeek	DeepSeek-V4-Pro, DeepSeek-R1 等
ollama	本地 Ollama	本地模型推理
openrouter	OpenRouter	多提供商聚合
groq	Groq	高速推理
zhipu	智谱 GLM	GLM-4 系列
moonshot	月之暗面	Kimi 系列
dashscope	阿里百炼	Qwen 系列
siliconflow	硅基流动	国产模型聚合
volcengine	火山引擎	豆包系列
lm_studio	LM Studio	本地桌面推理
vllm	vLLM	高性能推理服务
openai_codex	OpenAI Codex	OAuth 登录（无需手动提供 Key）
github_copilot	GitHub Copilot	OAuth 登录

提示

provider 字段设为 "auto" 时，astro-one 会根据模型名称中的关键词自动匹配提供商。例如，模型名 "anthropic/claude-opus-4-7" 会自动匹配到 Anthropic 提供商。

20 modelPresets —模型预设

模型预设允许用户定义多组模型配置，并可通过 `/model <name>` 命令快速切换。

```
{
  "modelPresets": {
    "fast": {
      "label": " 快速模式",
      "model": "openai/gpt-4o-mini",
      "provider": "openai",
      "maxTokens": 4096,
      "contextWindowTokens": 131072,
      "temperature": 0.3
    },
    "smart": {
      "label": " 深度思考",
      "model": "anthropic/claude-opus-4-7",
      "provider": "anthropic",
      "maxTokens": 8192,
      "contextWindowTokens": 200000,
      "temperature": 0.1,
      "reasoningEffort": "high"
    },
    "local": {
      "label": " 本地模型",
      "model": "qwen3.5:27b",
      "provider": "ollama",
      "maxTokens": 4096,
      "contextWindowTokens": 32768
    }
  }
}
```

21 channels —渠道配置

```
{
  "channels": {
    "sendProgress": true,          // 发送 Agent 进度到渠道
  }
}
```

```
"sendToolHints": false,      // 发送工具调用提示
"showReasoning": true,       // 显示模型推理过程
"sendMaxRetries": 3,         // 消息投递最大重试次数
"transcriptionProvider": "groq", // 语音转文字后端

"telegram": {
  "enabled": true,
  "botToken": "12345:xxx",
  "allowFrom": ["*"]
},
"feishu": {
  "enabled": true,
  "appId": "cli_xxxxx",
  "appSecret": "xxxxx",
  "allowFrom": ["ou_xxx"]
},
"discord": {
  "enabled": true,
  "botToken": "xxxxx",
  "allowFrom": ["*"]
},
"websocket": {
  "enabled": true,
  "port": 18791
}
}
```

警告

allowFrom 字段为空列表 [] 表示拒绝所有用户访问。设置为 ["*"] 表示允许所有用户。请在生产环境中仔细配置访问控制。

22 tools 一工具配置

```
{
  "tools": {
    "web": {
      "searchEnable": true,
      "fetchEnable": true
    }
  }
}
```

```
    },
    "exec": {
      "enable": true,
      "timeout": 60,
      "restrictToWorkspace": false,
      "denyPatterns": ["rm -rf /", "shutdown"]
    },
    "my": {
      "enable": true,
      "allowSet": true
    },
    "imageGeneration": {
      "enable": false
    },
    "restrictToWorkspace": false,
    "ssrfWhitelist": [],
    "mcpServers": {
      "my-server": {
        "type": "stdio",
        "command": "python",
        "args": ["-m", "my_mcp_server"],
        "toolTimeout": 30,
        "enabledTools": ["*"]
      }
    }
  }
}
```

23 环境变量

astro-one 支持通过环境变量覆盖配置。前缀为 NANOBOT__，使用双下划线分隔嵌套层级。

环境变量	说明
NANOBOT__AGENTS__DEFAULTS__MODEL	默认模型
NANOBOT__PROVIDERS__OPENAI__API_KEY	OpenAI API 密钥
ASTRO_ONE_MAX_CONCURRENT_REQUESTS	最大并发请求数（默认 3）
ASTRO_ONE_LLM_TIMEOUT_S	LLM 调用超时秒数（默认 300）
ASTRO_ONE_STREAM_IDLE_TIMEOUT_S	流式空闲超时秒数

表 11: 常用环境变量

航天专业工具

astro-one 内置三个航天垂直领域的专业 AI 推理工具，均位于 tools/ 目录下，通过 astro_one/agent/tools/ 中的 Python 包装器注册到 Agent 工具链中。

```
tools/
├── mlf/                                # 液体状态机轨道机动检测
│   ├── src/mlf.py                     # LSM 核心模块
│   ├── src/predictor.py               # 推理预测器
│   ├── models/                       # 预训练模型
│   └── data/                          # demo 数据
├── iod/                               # Transformer 轨道初定
│   ├── src/models.py                 # Transformer 模型定义
│   ├── src/predictor.py              # 推理预测器
│   ├── models/                       # Encoder/Decoder 模型
│   └── scalers/                      # 数据标准化器
└── orbin/                             # Informer 轨道预测
    ├── src/1021_best.py               # Informer 模型
    ├── src/predictor.py               # 集成预测器
    ├── models/                       # 三模型集成
    └── data/                          # 真实观测数据
```

图 1: 航天工具目录结构

24 MLF — 轨道机动检测

24.1 功能概述

基于液体状态机（Liquid State Machine）的卫星轨道机动检测模型。从卫星轨道参数序列中检测是否发生轨道机动，输出机动概率和预测机动时间。

24.2 Agent 工具名

mlf_maneuver_detection

24.3 输入数据格式

CSV 文件，包含 23 维特征列（卫星 ID + 轨道参数包括倾角、RAAN、偏心率、升交点赤经、半长轴、变轨前后参数等）。

24.4 输出格式

- satellite_id: 卫星 ID
- prediction: 机动判定结果（maneuver/no_maneuver）
- maneuver_prob: 机动概率（0-1）
- no_maneuver_prob: 未机动概率
- maneuver_time_days: 预测机动时间（天）

24.5 使用示例

```
# 通过 CLI 调用 Agent
astroone agent -m " 使用 MLF 工具分析 tools/mlf/data/demo_data.csv
→ 中的轨道数据，检测是否有卫星进行了轨道机动"

# Python SDK
from astro_one.astro_one import AstroOne
bot = AstroOne.from_config()
result = await bot.run(" 用 mlf_maneuver_detection 工具检测
→ tools/mlf/data/demo_data.csv")
```

24.6 军事分析输出

工具会自动输出包括以下内容的态势分析：

- 态势等级：高/中/低风险评估
- 概率分层：高置信度（>0.8）、中置信度（0.5-0.8）、低置信度（0.3-0.5）
- 优先处置清单：按机动概率排序的重点目标
- 意图研判：轨道转移、规避/反跟踪、抵近操作准备三类假设复核建议

25 IOD — 轨道初定

25.1 功能概述

基于 **Transformer** 编码器-解码器架构的初始轨道确定（Initial Orbit Determination）模型。从地基观测数据（时间、站址、方向向量）预测卫星的 ECI 位置和速度。

25.2 Agent 工具名

```
iod_orbit_determination
```

25.3 输入数据格式

CSV 文件，包含以下列：

- Relative Time (s)：相对时间
- Observer Longitude / Latitude / Altitude：观测者地理坐标
- Observer ECI X / Y / Z：观测者地心惯性坐标
- Direction Vector X / Y / Z：方向向量（视线方向）

25.4 输出格式

- Direction Vector X / Y / Z：预测的方向向量
- Satellite ECI X / Y / Z (km)：卫星位置
- Satellite Velocity X / Y / Z (km/s)：卫星速度

25.5 军事分析输出

- 轨道类型判断：LEO / MEO / HEO / GEO 自动分类
- 倾角估计：用于判断卫星纬向覆盖能力
- 目标保持风险：基于高度变化幅度和速度离散度的风险评估

26 Orbin — 轨道根数预测

26.1 功能概述

基于 **Informer** 模型的轨道六根数预测模型，使用**三模型集成**提升预测精度。从观测弧段数据预测卫星未来的 ECI 位置和速度。

26.2 Agent 工具名

orbin_orbit_prediction

26.3 输入数据格式

CSV 文件，包含以下列：

- Time_UTCG_：UTC 时间
- Azimuth_deg_：方位角（度）
- Elevation_deg_：高度角（度）
- Range_km_：距离（km）
- eci_x_m, eci_y_m, eci_z_m：卫星 ECI 位置（米）

26.4 输出格式

- x_km_pred, y_km_pred, z_km_pred：预测位置（km）
- vx_km/s_pred, vy_km/s_pred, vz_km/s_pred：预测速度（km/s）
- 可选：*_true 列（用于精度对比）

26.5 军事分析输出

- 轨道周期：预估轨道周期
- 预测可信度：基于高度/速度离散度的置信度评估
- 跟踪优先级：一级/二级/三级分类，指导资源分配
- 碰撞/抵近筛查建议：同轨道面、相近高度目标交叉检查

27 航天工具联合使用

三个工具可以串联使用，形成完整的空间态势感知流水线：

1. **IOD** 对原始观测进行轨道初定 → 获得卫星初轨状态
2. **Orbin** 基于初轨进行预测外推 → 获得未来轨道位置
3. **MLF** 对轨道参数序列进行机动检测 → 识别异常机动行为

```
astroone agent -m "
```

请按以下流程进行空间态势分析：

1. 使用 iod_orbit_determination 对 tools/iod/data/demo_input.csv
→ 进行轨道初定
2. 使用 orbin_orbit_prediction 基于初定结果进行轨道预测
3. 使用 mlf_maneuver_detection 检测是否有轨道机动
4. 综合以上结果，给出空间态势评估报告

"

聊天渠道

28 渠道概述

astro-one 支持 15+ 聊天平台，所有渠道通过统一的 BaseChannel 接口接入。渠道配置在 config.json 的 channels 部分。

渠道名	配置键	依赖包
飞书	feishu	lark-oapi
Telegram	telegram	python-telegram-bot
Discord	discord	内置 aiohttp
Slack	slack	slack-sdk
QQ	qq	qq-botpy
WhatsApp	whatsapp	内置 websocket-client
微信	weixin	pycryptodome
钉钉	dingtalk	dingtalk-stream
企业微信	wecom	wecom-aibot-sdk-python
Matrix	matrix	matrix-nio
Signal	signal	可选
Email	email	内置 SMTP/IMAP
MS Teams	msteams	PyJWT
MoChat	mochat	内置 WebSocket
WebSocket	websocket	websockets（Web UI 后端）

29 飞书渠道配置详解

飞书（Feishu/Lark）是目前最完善的企业级渠道之一。

```
{
  "channels": {
    "feishu": {
      "enabled": true,
      "appId": "cli_XXXXXXXXXXXX",
      "appSecret": "XXXXXXXXXXXXXXXXXXXXXXXXXXXX",
      "verificationToken": "XXXXXXXXXXXX",
      "allowFrom": ["ou_xxx", "ou_yyy"],
    }
  }
}
```

```
        "streaming": true
    }
}
```

29.1 飞书特有功能

- **消息卡片**：支持富文本卡片回复
- **流式输出**：支持 `send_delta` 实现打字机效果
- **推理展示**：支持推理过程折叠/展开（低强调 UI）
- **代码块**：长代码自动包装为飞书代码块
- **表格拆分**：大表格自动拆分为多条消息
- **Markdown 渲染**：适配飞书 Markdown 语法

30 渠道开发

如需开发新的渠道插件，继承 `BaseChannel` 并实现以下方法：

```
from astro_one.channels.base import BaseChannel

class MyChannel(BaseChannel):
    name = "my_channel"
    display_name = " 我的渠道"

    async def start(self) -> None:
        """ 启动渠道，开始监听消息 """
        # 1. 连接到平台
        # 2. 循环接收消息
        # 3. 调用 self._handle_message() 转发到总线
        ...

    async def stop(self) -> None:
        """ 停止渠道，清理资源 """
        ...

    async def send(self, msg: OutboundMessage) -> None:
        """ 发送消息到平台 """
        ...
```

LLM 提供商深度配置

31 提供商匹配机制

astro-one 的提供商选择使用以下优先级：

1. 显式 **provider** 前缀：如模型名 anthropic/claude-opus-4-7
2. 关键词匹配：如模型名含"claude" 匹配 Anthropic
3. 本地提供商回退：当有可用 apiBase 时
4. 已配置的 **API Key** 提供商：按顺序匹配

32 常用提供商配置示例

32.1 OpenAI

```
{
  "providers": {
    "openai": {
      "apiKey": "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx",
      "apiBase": "https://api.openai.com/v1",
      "apiType": "chat_completions"
    }
  },
  "agents": {
    "defaults": {
      "model": "openai/gpt-4o",
      "provider": "auto"
    }
  }
}
```

32.2 Anthropic Claude

```
{
  "providers": {
    "anthropic": {
      "apiKey": "sk-ant-api03-xxxxxxxxxxxxxxxxxxxxxxxx",
      "extraHeaders": {
        "anthropic-beta": "claude-code-20250529"
      }
    }
  },
}
```

```
"agents": {
  "defaults": {
    "model": "anthropic/claude-opus-4-7",
    "provider": "auto"
  }
}
```

32.3 本地 Ollama

```
{
  "providers": {
    "ollama": {
      "apiBase": "http://localhost:11434"
    }
  },
  "agents": {
    "defaults": {
      "model": "qwen3.5:27b",
      "provider": "ollama"
    }
  }
}
```

32.4 AWS Bedrock

```
{
  "providers": {
    "bedrock": {
      "apiKey": "AKIAxxxxxx",
      "region": "us-east-1",
      "profile": "default"
    }
  },
  "agents": {
    "defaults": {
      "model": "bedrock/anthropic.claude-opus-4-7",
      "provider": "auto"
    }
  }
}
```

```
}  
}
```

32.5 DeepSeek

```
{  
  "providers": {  
    "deepseek": {  
      "apiKey": "sk-xxxxxxxxxxxxx",  
      "apiBase": "https://api.deepseek.com/v1"  
    }  
  },  
  "agents": {  
    "defaults": {  
      "model": "deepseek/deepseek-v4-pro",  
      "provider": "auto"  
    }  
  }  
}
```

33 备选模型（Fallback）

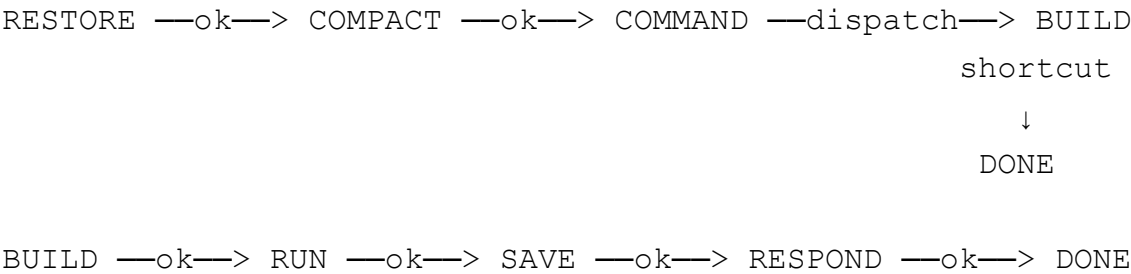
当主模型不可用时，自动切换到备选模型：

```
{  
  "agents": {  
    "defaults": {  
      "model": "anthropic/claude-opus-4-7",  
      "fallbackModels": [  
        "smart",  
        {  
          "model": "openai/gpt-4o",  
          "provider": "openai",  
          "contextWindowTokens": 131072  
        }  
      ]  
    }  
  }  
}
```

Agent 核心机制

34 状态机处理流程

Agent 的消息处理采用 8 阶段状态机：



状态	职责	下一状态
RESTORE	恢复 checkpoint、提取文档、记录日志	COMPACT
COMPACT	检查并执行会话自动压缩	COMMAND
COMMAND	检查内置命令（/new、/model 等）或模型指令	BUILD 或 DONE
BUILD	构建 prompt、加载历史/记忆/技能、设置工具上下文	RUN
RUN	调用 LLM 执行 Agent 循环、工具调用	SAVE
SAVE	保存会话消息、清除 checkpoint、触发后巩固	RESPOND
RESPOND	组装出站消息、处理流式标记	DONE
DONE	结束处理	—

表 13: 状态机各阶段说明

35 上下文构建

ContextBuilder 负责为每次 LLM 调用构建完整的提示词，包含：

- 1. 身份声明：定义 Agent 名称、开发者、能力范围
- 2. **Bootstrap 文件**：加载工作空间中的 AGENTS.md、SOUL.md、USER.md、TOOLS.md
- 3. 工具契约：工具使用规范和约束
- 4. 长期记忆：从 MEMORY.md 加载关键信息
- 5. 活跃技能：always 类型的技能内容
- 6. 技能摘要：按需激活的技能列表
- 7. 近期历史：从 history.jsonl 加载近期对话摘要
- 8. 会话摘要：被压缩的历史归档摘要
- 9. 运行时上下文：当前时间、渠道、聊天 ID、工作空间路径

36 记忆系统

astro-one 采用三级记忆架构：

层级	组件	功能
L1 — 工作记忆	Session 消息列表	当前对话的完整消息历史
L2 — 短期记忆	Consolidator	将旧消息 LLM 摘要后写入 history.jsonl
L3 — 长期记忆	Dream	Cron 定时分析历史，更新 MEMORY.md / SOUL.md / USER.md

表 14: 三级记忆架构

36.1 Dream 记忆处理器

Dream 是一个两阶段的记忆巩固处理器：

1. **Phase 1** — 分析：LLM 分析历史对话，提取关键信息、用户偏好、项目状态
2. **Phase 2** — 执行：使用 AgentRunner + 文件编辑工具，增量更新 MEMORY.md、SOUL.md、USER.md 和技能文件

Dream 默认每 2 小时触发一次，可通过配置调整：

```
{
  "agents": {
    "defaults": {
      "dream": {
        "intervalH": 4,
        "maxBatchSize": 30,
        "maxIterations": 12
      }
    }
  }
}
```

37 子 Agent (SubAgent)

SubagentManager 允许 Agent 生成后台子任务进行并发处理。子 Agent 拥有：

- 独立的消息上下文和工具集
- 实时状态追踪（初始化 → 等待工具 → 完成）
- 会话内的事件注入机制

- 可配置的最大并发数

38 会话管理

- 存储格式: JSONL (每行一条 JSON)
 - 存储位置: workspace/sessions/<key>.jsonl
 - 会话键: 默认 <channel>:<chat_id>, 支持 unified session 模式
 - 自动压缩: 达到上下文窗口限制时自动触发 Consolidator
 - 空闲回收: 配置 sessionTtlMinutes 后, 空闲会话自动压缩
 - 故障恢复: 支持 checkpoint 恢复中断的会话和崩溃后的 pending user turn
- 工具系统

39 内置工具列表

工具	Agent 名	功能
Shell 执行 文件系统	exec	执行 shell 命令, 支持长命令会话管理
	read_file	读取文件内容
	edit_file	精确字符串替换编辑
	write_file	写入/覆写文件
	list_dir	列出目录内容
	grep	正则搜索文件内容
	find_files	按 glob 模式搜索文件
Web 工具	web_search	网络搜索
	web_fetch	获取网页内容转换为 Markdown
MCP	动态注册	Model Context Protocol 工具
消息发送	message	向特定聊天渠道发送消息
定时任务	cron_*	创建/删除/列出定时任务
自我配置	my_*	运行时修改 Agent 配置
图片生成	image_gen	AI 图片生成
CLI 应用	cli_apps	调用外部 CLI 工具

40 Shell 工具详解

exec 工具支持:

- 命令超时: 可配置超时时间 (默认 60 秒)
- 会话管理: 支持后台长任务 (ExecSession), 可后续获取输出
- 工作目录限制: 可配置是否限制在 workspace 内

- 模式过滤: deny_patterns / allow_patterns 支持正则表达式过滤危险命令
- 输出截断: 大输出自动截断

41 MCP 协议支持

astro-one 支持 MCP（Model Context Protocol）协议的三种传输方式:

传输类型	适用场景	配置示例
stdio	本地进程	<pre>{"command": "python", "args": ["-m", "server"]}</pre>
sse	HTTP SSE	<pre>{"url": "http://localhost:3000/sse"}</pre>
streamableHttp	HTTP 流式	<pre>{"url": "http://localhost:3000/mcp"}</pre>

表 16: MCP 传输方式

42 工具扩展

创建自定义工具只需继承 Tool 基类:

```
from astro_one.agent.tools.base import Tool
from typing import Any

class MyCustomTool(Tool):
    @property
    def name(self) -> str:
        return "my_custom_tool"

    @property
    def description(self) -> str:
        return " 我的自定义工具  —描述工具的功能"

    @property
    def parameters(self) -> dict[str, Any]:
        return {
            "type": "object",
            "properties": {
                "input_path": {
                    "type": "string",
```

```
        "description": " 输入文件路径"
    }
},
    "required": ["input_path"]
}

async def execute(self, input_path: str, **kwargs) -> str:
    # 工具逻辑
    return f" 处理完成: {input_path}"
```

安全机制

43 SSRF 防护

Web 工具（web_search、web_fetch）内置完整的 SSRF（服务端请求伪造）防护：

- **DNS 解析检查**：所有 URL 的域名先解析为 IP
- **内网 IP 拦截**：阻止访问 10.x、172.16-31.x、192.168.x、127.x、169.254.x 等私有/保留地址
- **Cloud Metadata 防护**：阻止 169.254.169.254 等实例元数据地址
- **白名单机制**：通过 tools.ssrfWhitelist 配置可信内网地址

44 Workspace 隔离

- **文件工具**：read_file、edit_file、write_file 支持限制在配置的 workspace 内
- **Shell 工具**：支持 restrictToWorkspace 模式（通过配置文件控制）
- **路径遍历防护**：检测 ../ 路径穿越尝试

45 命令注入防护

Shell 工具的 denyPatterns 支持正则表达式过滤危险命令模式：

```
{
  "tools": {
    "exec": {
      "denyPatterns": [
        "rm\\s+-rf\\s+/",
        "mkfs\\. ",
        "dd\\s+if=",
```

```
        ":( ){ :|:& }|:",
        "chmod\\s+777"
    ]
}
}
```

46 访问控制

- **渠道级**：每个渠道的 allowFrom 白名单
- **配对机制**：未授权用户收到配对码而非被静默忽略
- **API 认证**：API 服务支持密钥认证

Docker 部署

47 快速部署

```
# 克隆项目
git clone https://github.com/astro-one/astro-one.git
cd astro-one

# 创建 .env 文件
cat > .env << 'EOF'
NANOBOT__PROVIDERS__ANTHROPIC__API_KEY=sk-ant-xxxxx
NANOBOT__AGENTS__DEFAULTS__MODEL=anthropic/claude-opus-4-7
EOF

# 启动 Gateway
docker compose up -d astro-one-gateway

# 查看日志
docker compose logs -f astro-one-gateway
```

48 docker-compose.yml 结构

```
services:
  astro-one-gateway:
    build: .
```

```
ports:
  - "18790:18790"    # Gateway 端口
  - "18791:18791"    # WebUI WebSocket 端口
volumes:
  - ~/.astro_one:/root/.astro_one  # 持久化配置和会话
  - ./tools:/app/tools              # 航天工具模型文件
environment:
  - NANOBOT__PROVIDERS__ANTHROPIC__API_KEY=${ANTHROPIC_KEY}
restart: unless-stopped
```

49 多实例部署

可以为不同渠道创建独立的配置和实例：

```
# 实例 1 — Telegram
astroone onboard \
  --config ~/.astro-one-telegram/config.json \
  --workspace ~/.astro-one-telegram/workspace

# 实例 2 — 飞书
astroone onboard \
  --config ~/.astro-one-feishu/config.json \
  --workspace ~/.astro-one-feishu/workspace

# 分别启动
astroone gateway --config ~/.astro-one-telegram/config.json &
astroone gateway --config ~/.astro-one-feishu/config.json &
```

技能系统

50 技能概述

技能（Skill）是 astro-one 的扩展机制，每个技能是一个包含 SKILL.md 文件的目录。SKILL.md 使用 YAML frontmatter 定义元数据和 markdown 正文作为 LLM 指令。

51 技能目录结构

```
workspace/skills/
```

```
├─ my-skill/  
│   └─ SKILL.md          # 技能定义 (YAML frontmatter + Markdown  
    ↳ 指令)  
└─ another-skill/  
    └─ SKILL.md
```

52 SKILL.md 格式

```
---  
name: my-skill  
description: 我的自定义技能 —帮助 Agent 理解特定领域知识  
type: on-demand          # always | on-demand  
priority: 10  
---  
  
# 我的技能  
  
当用户询问关于 xxx 的问题时，你应该：  
  
1. 首先检查 ...  
2. 使用 ... 工具获取数据  
3. 按照 ... 格式回复
```

53 内置技能

astro-one 内置的技能位于 `astro_one/skills/` 目录下，包括但不限于：

- **github**: GitHub 仓库操作
- **weather**: 天气查询
- **tmux**: tmux 会话管理
- **cron**: 定时提醒
- **skill-creator**: 技能创建助手
API 服务

54 OpenAI 兼容 API

astro-one 提供 OpenAI 兼容的 API 端点，可直接替换 OpenAI SDK 的后端。

54.1 配置

```
{
  "api": {
    "host": "127.0.0.1",
    "port": 8900,
    "timeout": 120.0
  }
}
```

54.2 端点

- POST /v1/chat/completions — Chat Completion
- GET /v1/models — 模型列表

54.3 调用示例

```
curl http://localhost:8900/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "anthropic/claude-opus-4-7",
  "messages": [{"role": "user", "content": "解释开普勒定律"}],
  "stream": false
}'
```

Benchmark 测评

55 AeroBench — 航天垂直领域基准测评

astro-one 附带 benchmark/AeroBench/ 专项测评工具，覆盖三项航天核心任务的量化评估。

55.1 运行方式

```
# 模拟模式（快速 baseline）
python -m benchmark.AeroBench.main --mode tool-sim
↪ --print-metrics
```

```
# 真实模型模式
python -m benchmark.AeroBench.main --mode tool-direct --device
↳ cpu --print-metrics

# 选择任务和样本数
python -m benchmark.AeroBench.main --mode tool-sim --tasks mlf
↳ iod --max-samples 500
```

55.2 测评任务

任务	核心指标	说明
MLF 机动检测	F1, ROC-AUC, 机动时间 MAE	评估机动检测的精确率和召回率
IOD 轨道初定	位置 RMSE, 方向余弦相似度	评估轨道确定的定位精度
Orbin 轨道预测	位置 RMSE, 倾角误差, 周期偏差	评估轨道预测的精度与稳定性

表 17: AeroBench 测评任务

常见问题与排错

56 安装问题

注意

Q: pip install 报错”Could not find a version...”

A: 确认 Python 版本 ≥ 3.11 。使用 `pip install -e .` 从源码安装。

注意

Q: 缺少某些渠道的依赖包

A: 渠道依赖是可选的，使用 `pip install "astro-one-ai[wecom,matrix]"` 安装特定渠道依赖。

57 配置问题

注意

Q: 启动报“No API key configured”

A: 检查 config.json 中对应 provider 的 apiKey 字段，或设置环境变量 NANOBOT__PROVIDERS__<PROVIDER>__API_KEY。

注意

Q: 模型调用返回“model not found”

A: 检查模型名称格式：provider 前缀（如 anthropic/）、模型版本号、provider 提供商是否支持该模型。

58 运行时间问题

注意

Q: Agent 循环卡住不返回

A: 可能是 LLM 调用超时。设置环境变量 ASTRO_ONE_LLM_TIMEOUT_S=120 缩短超时，或检查网络连接。

注意

Q: 工具调用返回“outside the configured workspace”

A: 已启用 workspace 隔离。将文件放在 workspace 目录下，或在配置中设置 restrictToWorkspace: false。

注意

Q: 会话恢复时出现重复消息

A: 这是 checkpoint 恢复机制的正常行为。checkpoint 基于消息指纹进行去重，重复消息会在下次迭代时自动对齐。

59 渠道问题

注意

Q: 飞书渠道收不到消息

A: 检查 `allowFrom` 白名单是否包含用户 ID, 确认 `appId/appSecret/verificationToken` 配置正确, 确认飞书应用已发布并订阅了相应事件。

注意

Q: Telegram Bot 不响应

A: 检查 Bot Token 是否正确, 确认已运行 `astroone channels login --provider telegram` 进行登录, 确认防火墙允许出站连接到 Telegram API。

60 航天工具问题

注意

Q: MLF/IOD/Orbin 工具报”模型加载失败”

A: 确认 `tools/<name>/models/` 目录下存在完整的模型文件 (`.pth/.pkl`), 确认 torch 版本与模型兼容, 确认当前工作目录为项目根目录。

注意

Q: 航天工具执行缓慢

A: 默认使用 CPU 推理。如需加速, 可在调用时指定 `device: "cuda"` (需要 NVIDIA GPU + CUDA)。

完整配置示例

A 生产环境配置参考

```
{
  "agents": {
    "defaults": {
      "workspace": "/data/astro-one/workspace",
      "model": "anthropic/claude-opus-4-7",
      "provider": "auto",
```

```
    "maxTokens": 8192,
    "contextWindowTokens": 200000,
    "temperature": 0.1,
    "maxToolIterations": 200,
    "maxConcurrentSubagents": 2,
    "maxToolResultChars": 16000,
    "timezone": "Asia/Shanghai",
    "sessionTtlMinutes": 120,
    "maxMessages": 120,
    "consolidationRatio": 0.5,
    "fallbackModels": ["fast"],
    "dream": {
      "intervalH": 4,
      "maxBatchSize": 30,
      "maxIterations": 12
    }
  },
  },
  "modelPresets": {
    "fast": {
      "label": "快速回退",
      "model": "openai/gpt-4o-mini",
      "provider": "openai",
      "maxTokens": 4096,
      "contextWindowTokens": 131072,
      "temperature": 0.3
    }
  },
  "providers": {
    "anthropic": {
      "apiKey": "sk-ant-xxxxxx"
    },
    "openai": {
      "apiKey": "sk-xxxxxx"
    }
  },
  "channels": {
    "sendProgress": false,
    "sendToolHints": false,
    "feishu": {
      "enabled": true,
```

```
    "appId": "cli_XXXXXX",
    "appSecret": "XXXXXX",
    "allowFrom": ["ou_admin"]
  },
  "websocket": {
    "enabled": true,
    "port": 18791
  }
},
"tools": {
  "exec": {
    "enable": true,
    "timeout": 120,
    "restrictToWorkspace": true
  },
  "restrictToWorkspace": true
},
"gateway": {
  "host": "0.0.0.0",
  "port": 18790,
  "heartbeat": {
    "enabled": true,
    "intervalS": 1800
  }
}
}
```

命令行速查卡

命令	说明
astroone onboard	初始化配置和工作空间
astroone agent	进入交互式对话模式
astroone agent -m	单次对话
"..."	
astroone agent --stream	流式输出交互模式
astroone agent --thinking	显示推理过程
astroone gateway	启动网关（加载所有渠道）
astroone status	查看运行状态

astroone channels	登录聊天渠道
login	
astroone provider	登录 LLM 提供商
login	
/model	查看当前模型（交互模式内）
/model <name>	切换模型（交互模式内）
/new	新建会话（交互模式内）
python -m	运行航天基准测评
benchmark.AeroBench.main	

环境变量速查卡

环境变量	说明
NANOBOT__AGENTS__DEFAULTS__MODEL	默认模型
NANOBOT__PROVIDERS__<name>__API_KEY	指定提供商的 API 密钥
NANOBOT__PROVIDERS__<name>__API_ENDPOINT	指定提供商的 API 端点
ASTRO_ONE_MAX_CONCURRENT_REQUESTS	最大并发请求数（默认 3）
ASTRO_ONE_LLM_TIMEOUT_S	LLM 调用超时秒数（默认 300）
ASTRO_ONE_STREAM_IDLE_TIMEOUT_S	流式空闲超时秒数
LITELLM_MODEL_COST_MAP	设为 false 禁用成本表加载